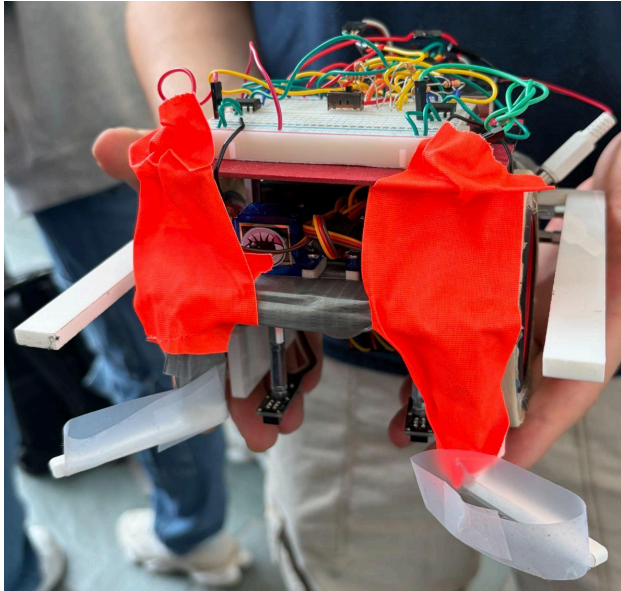


Robot Design and Strategy Overview



The mechanical design of the robot involves two arms that funnel cubes under the main frame of the robot. These arms were meant to actuate into open and closed positions, but this function never got implemented into the final strategy for competition. The funnel would allow a broader range of cubes to be acquired and secure them under the robot frame so they would not be lost. To hold cubes under the robot the wheels had to be bigger to make room under the frame. The larger

wheels also allowed for faster movement that would allow the robot to beat other teams to the middle, where the cubes start. The raising of the frame left gaps on the sides. To fix this issue side walls were made that would trap cubes under the frame. These parts can be seen in Appendix C along with other minor and unused parts.

The entire robot's electrical circuitry was controlled by an Arduino Uno. The driving motors' directions were controlled by external H-bridges. The arduino would also control the input settings of the color sensor so that it could properly differentiate between blue and yellow. It would then read outputs from the color sensor and two QTI sensors to get information about the field position. The QTI sensors were on the left and right sides of the robot so it would know which way to turn if the black border was detected. A detailed circuit diagram is in appendix B showing how each of these are wired.

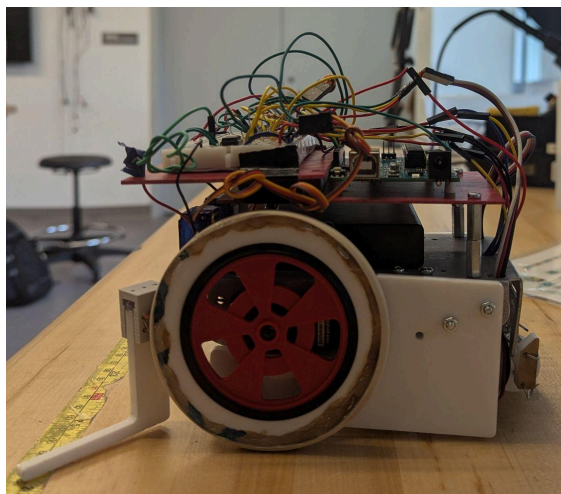
The software included 5 functions that would control the driving motors: drive forward, drive backward, turn left, turn right, and stop. Each of these could be called for a duration of time that would control how each would run for. A special delay function, called `del`, had to be made as using `_delay_ms()` with a local variable breaks the arduino code. The software would also measure the period returned by

the color sensor using timer 1 on the arduino and interpret it as blue or yellow. The QTI sensors were also linked to interrupts that would act when the black border was detected. These were the basic functions that allowed the strategy to be implemented.

The strategy of the robot was to drive at an angle to the left and reach the color border as close to the black border as possible. The robot would then turn so that it was parallel with the border and drive along it, collecting cubes as it went. Once it reached the black border it would enter an autonomous mode where it would drive around randomly staying in the black border to try to collect any cubes still available.

Design Process and Reflection

We began by assembling the basic model of the robot using the provided material along with parts in our kits. We used the provided CAD model of a basic robot provided on Canvas in order to create a baseline shape. Then, taking inspiration from clips and photos we'd seen of previous years' robots, we began discussing the design changes we could make to optimize the cube collection.



We wanted to primarily collect cubes underneath our robot's main body, so we made the decision to extend the wheel diameter to create this space. In order to contain the cubes once they were under the robot, we added 'walls' along the sides that enclosed the space under the robot, and added a piece to lift the ball at the back so that it would be level with the new enlarged wheels. To enlarge the wheels themselves, we laser cut acrylic rings that fit around the default wheels, and used rubber bands and

hot glue to add traction to the acrylic surface.

Originally, our QTI and color sensor were placed at the back of the robot. However, once we were able to get the sensors working, we realised that this

positioning meant the robot was very delayed in sensing the black border and would drive off the map before it was able to correct its trajectory. Therefore, we moved the QTI sensors to the front, which reduced the issue.

We had originally decided to have the doors open and close controlled by servo motors. We worked on this in lab and although we were able to achieve the desired motion, we found that it didn't really add much to the strategy when we tested our robot, and it was unreliable — often getting stuck. We decided to remove the opening and closing doors idea and we hot glued the hinge of each 'door' to fix them in place.

Competition Analysis

The team is overall proud of the robot's performance during the competition. We were able to win some battles, and we are satisfied with that. A big unexpected problem was that the robot was able to collect cubes in the beginning but once it reached the black border, the robot often dropped the cubes outside of the border due to the arms extending over the black border. This also happened due to the robot's rotation at the border. Another unexpected problem was that the robot suddenly stopped after moving forward a bit during one of the battles. Overall, we would say that the base idea of the robot was great since it was still able to collect more cubes in some battles due to the space below the robot and the arms.

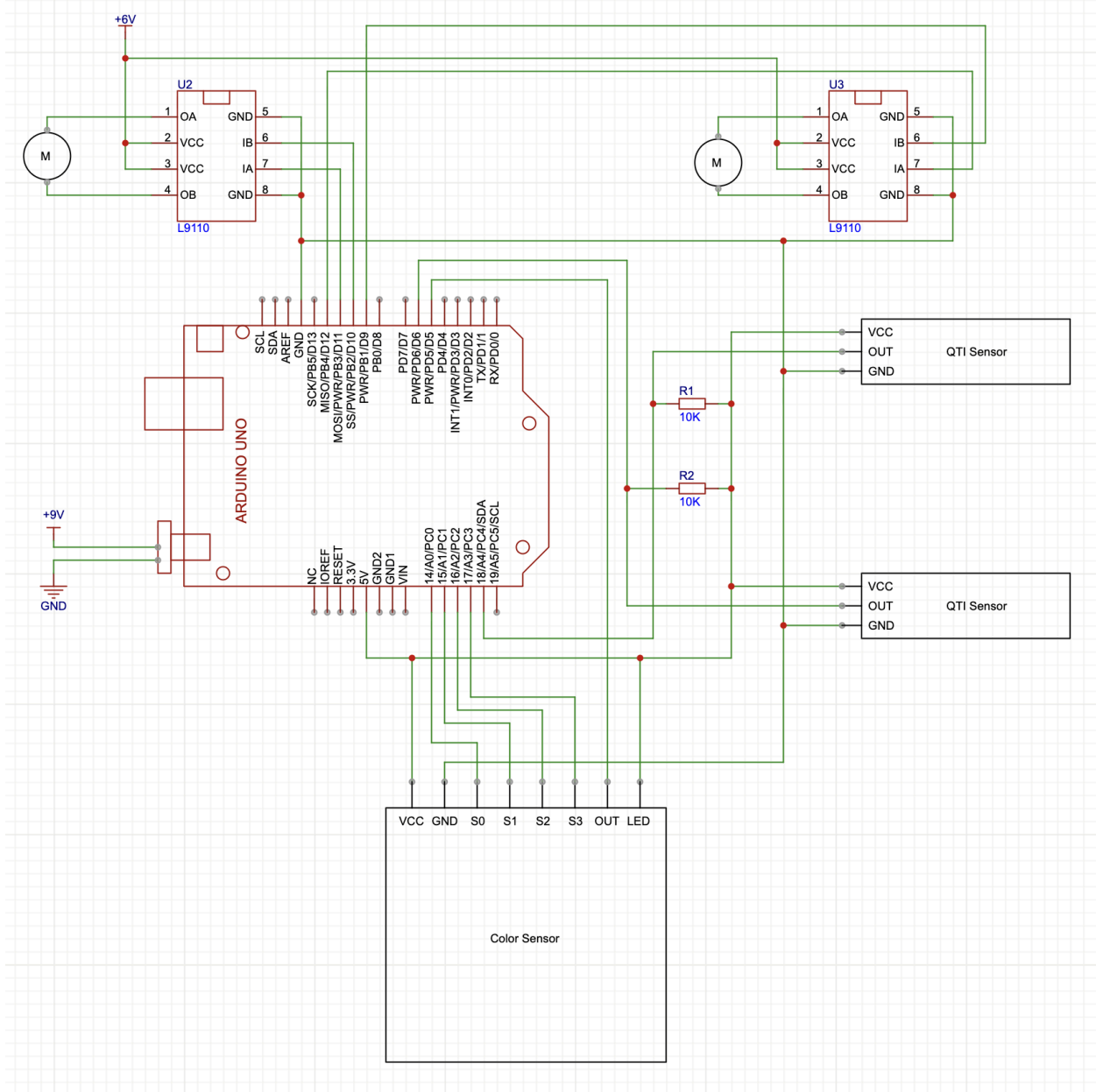
Conclusions

If we were to do this project using the same constraints and budget, we would focus more on the mechanical design of the robot so that the cubes don't leave the perimeter of our robot. We would also try to find a different strategy for the coding part by perhaps making the robot completely stop at the black border after encountering it a number of times instead of always making it turn around 180 degrees. Regardless, we are proud that we completed the milestones on time and won some battles during the competition. For students doing the project next year, we would tell them to make sure to complete the milestones early, communicate well with your team, and most importantly, just have fun!

Appendix A: Bill of Materials

Part Name	Quantity	Price Per Part (Dollars)	Total (Dollars)
Servo Horns	2	4.50	9.00
Hot Glue	1 (Stick)	0.25	0.25
Acrylic Sheet	1	5.00	5.00
Side Walls	2	1.12	2.24
Back Walls	1	1.15	2.30
Left Arm	1	0.93	0.93
Right Arm	1	0.88	0.88
Side Arms	2	1.30	2.60
Ball Mount	2	0.58	1.16
Servo Mounts	2	0.61	1.22
Wheel Extensions	2	1.22	2.44
Arms Attachment	2	1.60	3.20
Sensor Mount	1	4.60	4.60
Back Piece	1	0.76	0.76
Tape		0.15	0.15
Total			36.48

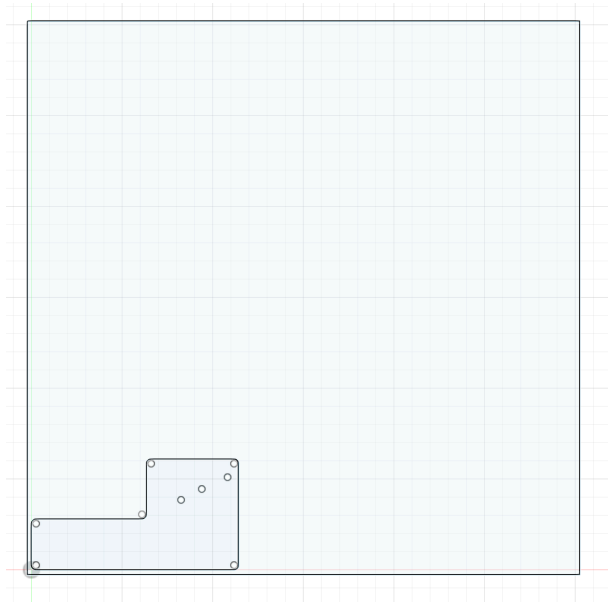
Appendix B: Circuit Diagram



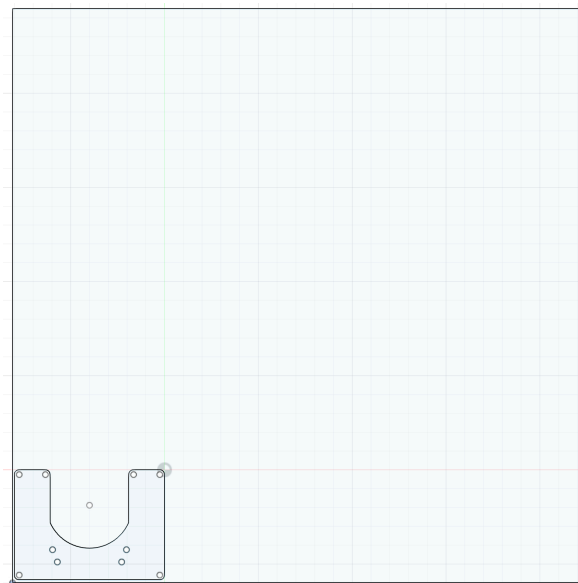
Appendix C: CAD files and drawings

Sketches of Parts for Laser Cutting:

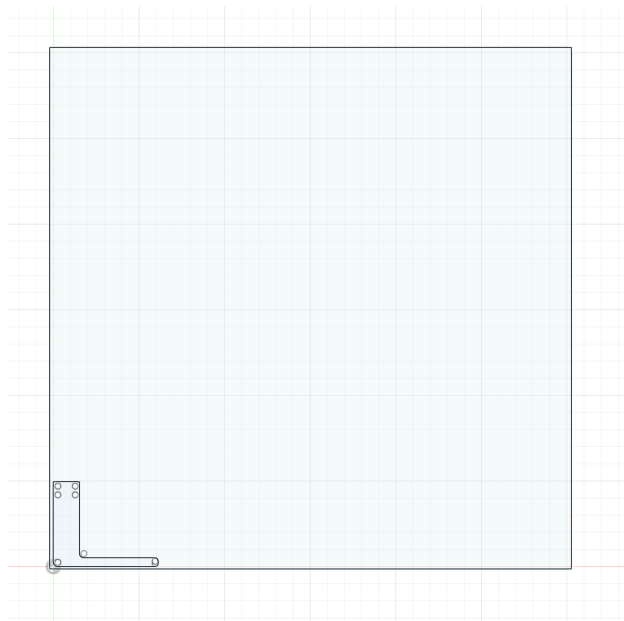
Side Wall: Perimeter was 12.40 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$1.12 each.



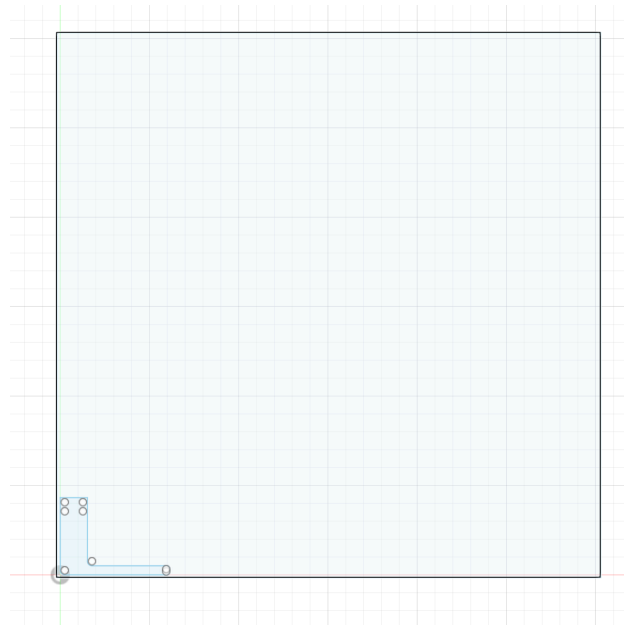
Back Wall (Unused): Perimeter was 13 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$1.15 each.



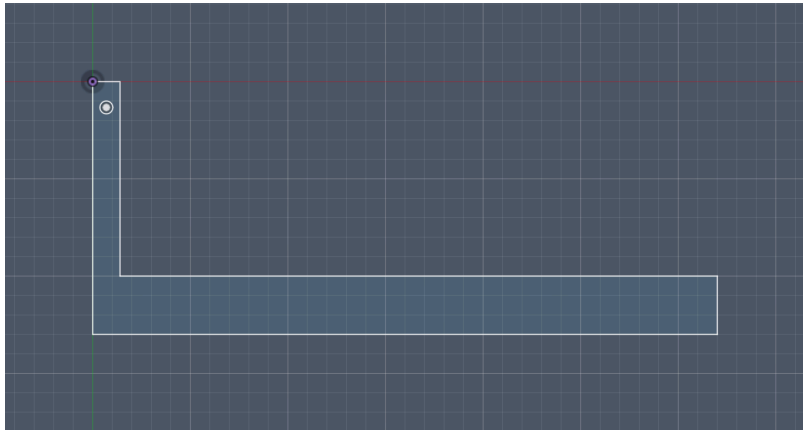
Left Arm: Perimeter was 8.6 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$0.93 each.



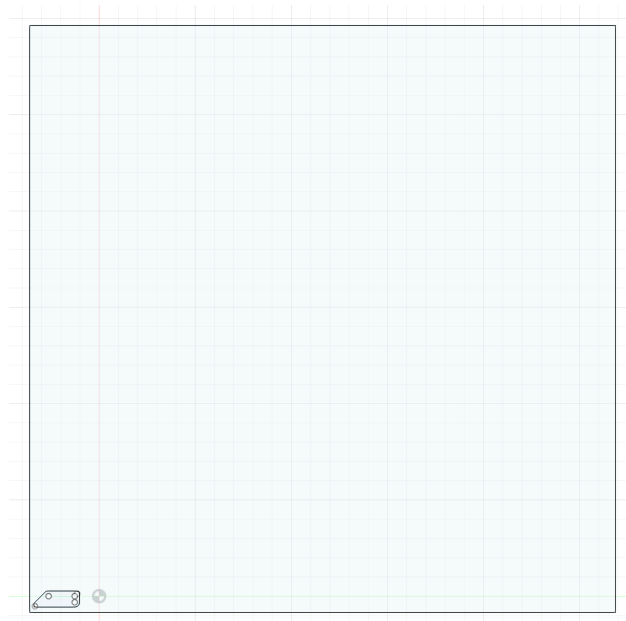
Right Arm: Perimeter was 7.6 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$0.88 each.



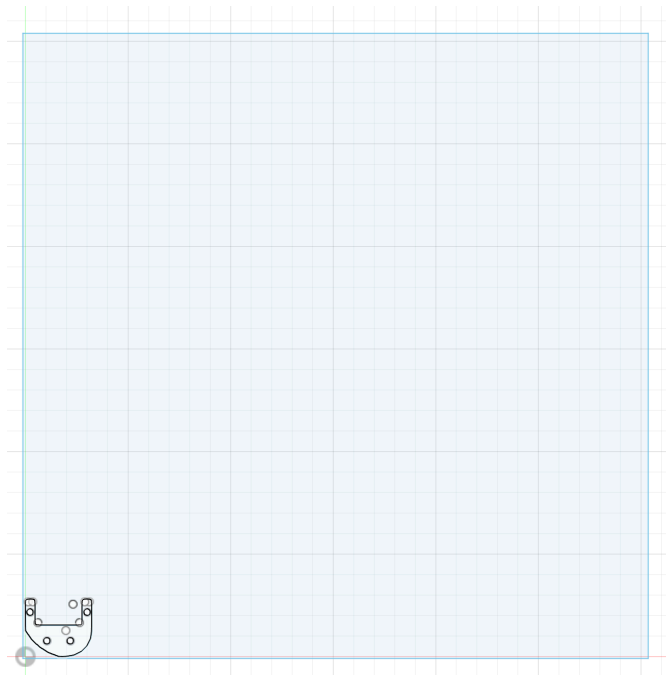
Side Arm: Perimeter was 16 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$1.30 each.



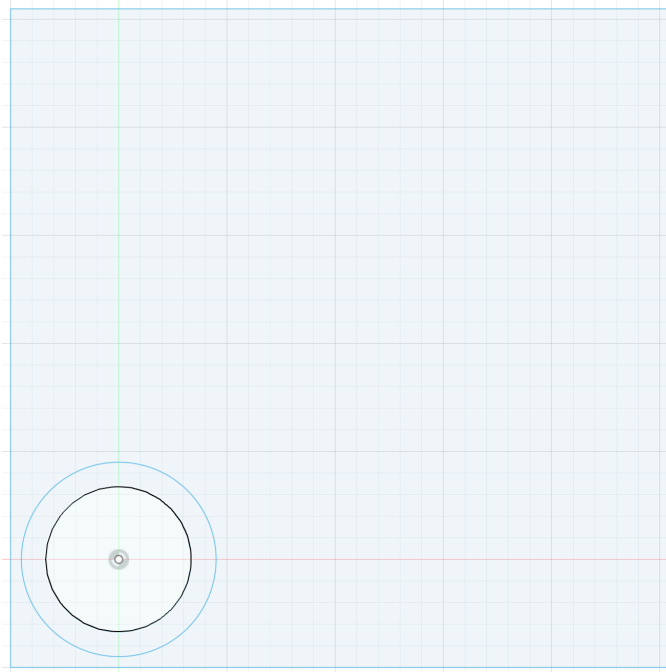
Ball Mount: Perimeter was 1.6 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$0.58 each.



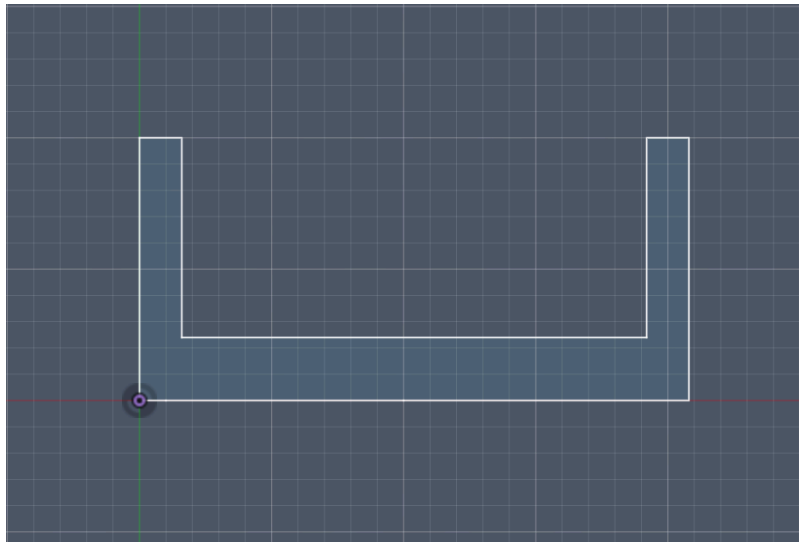
Servo Mount: Perimeter was 2.2 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$0.61 each.



Wheel Extensions: Perimeter was 14.4 inches at \$0.05 per inch and \$0.50 per part that means for each part they are \$1.22 each.

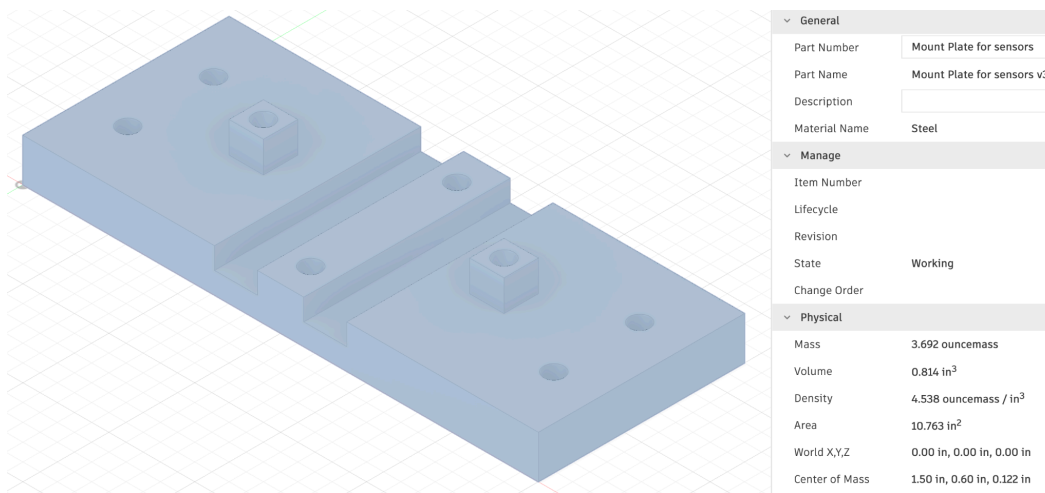


Back piece: Perimeter was 15.1 inches at \$0.05 per inch that means for each part they cost \$0.76 each.

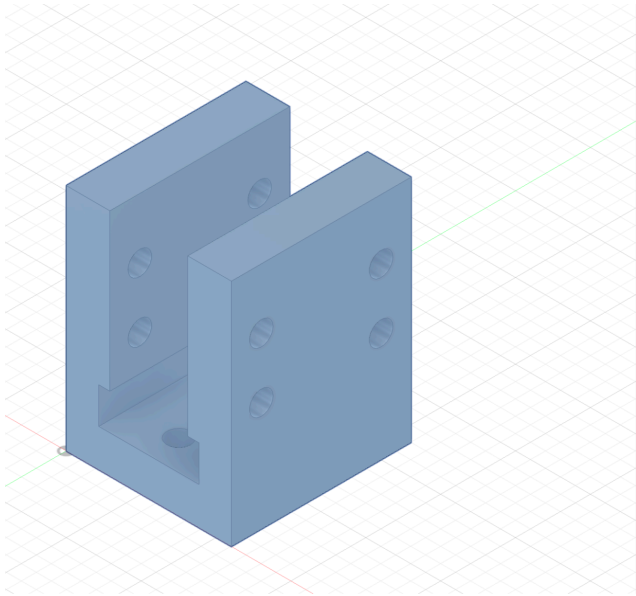


Models of Parts that were 3D printed:

Sensor Mount: Printed with PLA with a density of 20.43 g/in^3 . That would mean that the part was 16.54 g at a price of \$1 a part and \$0.40 a gram the total price would be \$6.61. However when weighted the part only weighed 9 grams likely due to the fact that the part is not solid PLA. This brings the total price down to \$4.60.



Arm Attachment: Printed with PLA with a density of 20.43 g/in^3 . That would mean that the part was 2.86 g at a price of \$1 a part and \$0.40 a gram the total price would be \$2.14. However when weighted the part only weighed 1.5 grams likely due to the fact that the part is not solid PLA. This brings the total price down to \$1.60.



General

Part Number	Arm Holder
Part Name	Arm Holder v5
Description	
Material Name	Steel

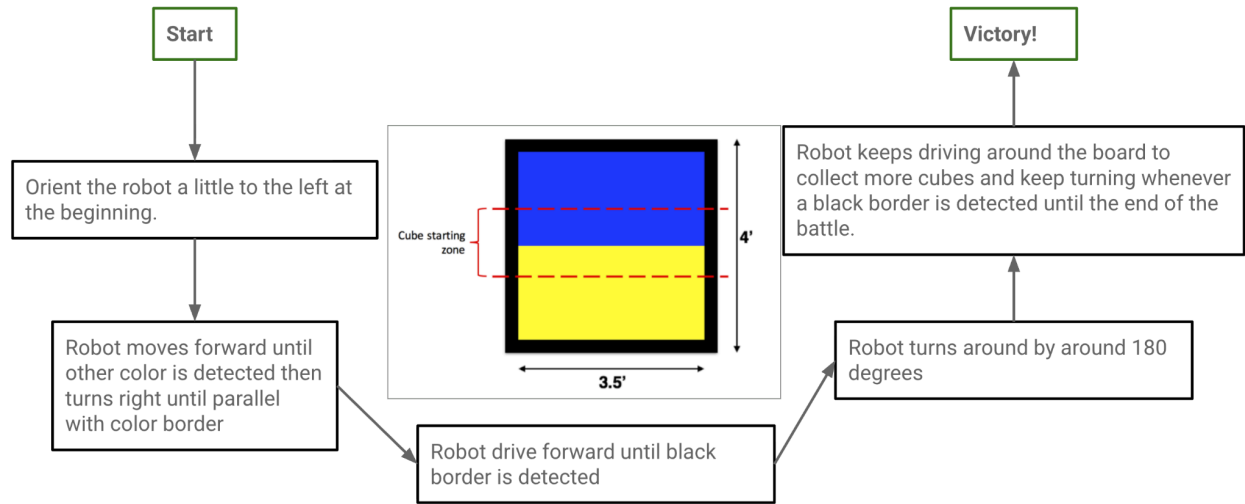
Manage

Item Number	
Lifecycle	
Revision	
State	Working
Change Order	

Physical

Mass	0.637 ouncemass
Volume	0.140 in ³
Density	4.538 ouncemass / in ³
Area	3.203 in ²
World X,Y,Z	0.00 in, 0.00 in, 0.00 in
Center of Mass	0.276 in, 0.30 in, 0.346 in

Appendix D: Flowchart



Appendix E: Code

```
// --- Global Variables ---

// "period" : stores the value of the output wave period (not necessarily in seconds)
int period; //microseconds
// "timer" : stores the value of TIMER1
int timer;
// "start_color" : helps indicate the beginning color of where the robot is at. Let 0
= BLUE, 1 = YELLOW
int start_color;
// "run" : indicates if the robot should move
int run;
// "lastState" : meant to help indicate falling vs. rising edge of interrupt
volatile uint8_t lastState = 0;

//All of our funtions and ISRs
// Drive functions with built-in delay
// Robot moves backward
void drive_backward_delay(int t){
    // code to control appropriate pins here:
    PORTB |= 0b00000110; // set pins 9 and 10 high
    del(t);              // wait for t milliseconds
    PORTB &= 0b11111001; // reset pins to low
}

// Robot moves forward
void drive_foward_delay(int t){ //454 ms is about 90 degrees
    PORTB |= 0b00011000; // set pins 11 and 12 high
    del(t);              // wait for t milliseconds
    PORTB &= 0b11100111; // reset pins to low
}

// Robot turns left
void drive_left_delay(int t){ // 167 ms is about an inch
    PORTB |= 0b00001010; // set pins 9 and 11 high
    del(t);              // wait for t milliseconds
    PORTB &= 0b11110101; // reset pins to low
}

// Robot turns right
void drive_right_delay(int t){
    PORTB |= 0b00010100; // set pins 10 and 12 high
    del(t);              // wait for t milliseconds
```

```

    PORTB &= 0b11101011; // reset pins to low
}

// Robot stops
void stop(int t){
    PORTB &= 0b11100001; // reset pins to low
    del(t);
}

// Our own delay function that will work with variable delays
void del(unsigned int t){
    for(int i = 0; i < t; i++){
        _delay_ms(1);
    }
}

// --- ISR: an interrupt function that resets and reads the timer value
ISR(PCINT2_vect)
{
    volatile int currentState = PIND & 0b00100000; // reads the state of pin 5
    // resets the timer to zero on a rising edge,
    if ((lastState == 0) && (currentState == 32)){
        TCNT1 = 0;
    }
    // and stores the timer value in a variable ("timer") on a falling edge
    if ((lastState == 32) && (currentState == 0)){
        timer = TCNT1;
    }
    // save pin state so it can be compared during the next interrupt call
    lastState = currentState;
}

// Initialize pins and enable interrupts on pins for color sensor
void initColor()
{
    // 1. Set S0, S1, S2, S3 as Outputs
    DDRC |= 0b00001111; // Set S0-S3 to pins A0-A3 respectively

    // 2. Set OUT (Pin 5) as Input
    DDRD &= 0b11011111;
}

```

```

// 3. Set Frequency Scaling to 20% (S0=H, S1=L)
PORTC |= 0b00000001;
PORTC &= 0b11111101;

// 4. Set Filter to Red (S2=L, S3=L)
PORTC &= 0b11110011;

// 5. Initialize Pin Change Interrupt for PCINT21 (Pin 5)
PCICR |= 0b00000100; // Enable Port D group interrupts

// 6. Initialize Timer 1
// set to normal mode
TCCR1A = 0b00000000;
TCCR1B = 0b00000000;

// set prescaler = 1
TCCR1B = 0b00000001;

sei();
}

// Initialize pins for the motors
void initMotor()
{
    DDRB |= 0b00011110; // Set pins 11 and 10 to outputs for left motor and pins 12 and
9 for right motor
    PORTB |= 0b00000000; // set motor pins to 0
}

// Initialize pins for the blackline sensors
void initBlackline()
{
    DDRC &= 0b11101111; // Set pin A4 for left blackline sensor
    DDRD &= 0b10111111; // Set pin 6 for right blackline sensor
}

// --- getColor: calculates the period of the sensor output wave and returns it as a
variable
int getColor()
{
    // a. Enable the specific bit for Pin Change Interrupt
    PCMSK2 |= 0b00100000;

```

```

    // b. Add a short delay (5 ~ 10 milliseconds)
    _delay_ms(50);
    // c. Disable the specific bit for pin change interrupt
    // (to prevent further interrupts until getColor is called again)
    PCMSK2 &= 0b00000000;

    // d. Return the period in units of time
    period = (timer * 2) / 16; // Convert the period measurement into microseconds
using the fact the one byte is 62.5 nano seconds
    return period; // return period in ms
}

int main(void){ // setup code that only runs once

    // enable serial port
    Serial.begin(9600);
    del(1000);

    // call initColor function
    initColor();

    // call initMotor function
    initMotor();

    // call initBlackline function
    initBlackline();

    run = 1; // let robot run in the beginning
    int start_color = 0; // indicates starting color robot is on. 1 is yellow and 2 is
blue. 0 is none
    int current_color; // indicates current color robot is on. 1 is yellow and 2 is blue.
0 is none

    getColor(); // reads the color the robot is on in the beginning
    if ((period > 20) && (period < 150)){
        start_color = 1; // robot is on yellow
    }
    else if ((period > 150) && (period < 400)) {
        start_color = 2; // robot is on blue
    }
}

```



```

}

while (run){
    getColor(); // reads the color the robot is on while it's moving
    bool right_black = (PIND & 0b00010000); // right QTI sensor detects black if pin 4
is high
    bool left_black  = (PINC & 0b00010000); // left QTI sensor detects black if pin A4
is high

    // if left QTI sensor detects black --> stop, turn right, and move forward
    if (left_black && !(right_black)){
        stop(1000);
        run = 0;
        drive_right_delay(600);
        run = 1;
        drive_foward_delay(100);
    }

    // if right QTI sensor detects black --> stop, turn left, and move forward
    else if (right_black && !(left_black)){
        stop(1000);
        run = 0;
        drive_left_delay(600);
        run = 1;
        drive_foward_delay(100);
    }

    // if both QTI sensors detect black --> stop, turn right, and move forward
    else if (left_black && right_black) {
        stop(1000);
        run = 0;
        drive_right_delay(900);
        run = 1;
        drive_foward_delay(100);
    }

    else{
        // Determine current color robot is on
        if ((period > 20) && (period < 150)) {
            current_color = 1; // Robot is on yellow
        }

        else if ((period > 150) && (period < 400)) {

```

```

    current_color = 2; // Robot is on blue
}
else {
    current_color = 0; // Robot is neither on blue or yellow
}

// Move forward while still on starting color
if (current_color == start_color) {
    drive_foward_delay(100);
}

// Turn right and move forward when color changes to the other one
else if ((current_color != start_color) && current_color != 0) {
    drive_right_delay(600);
    drive_foward_delay(100);
    while (run){
        getColor(); // Robot detects current color it is on
        bool right_black = (PIND & 0b00010000); // right QTI sensor detects black if
pin 4 is high
        bool left_black = (PINC & 0b00010000); // left QTI sensor detects black if
pin A4 is high
        drive_foward_delay(100); // Robot moves forward

        // if left QTI sensor detects black --> stop, turn right, and move forward
        if (left_black && !(right_black)){
            stop(1000);
            run = 0;
            drive_right_delay(600);
            run = 1;
            drive_foward_delay(100);
        }

        // if right QTI sensor detects black --> stop, turn left, and move forward
        else if (right_black && !(left_black)){
            stop(1000);
            run = 0;
            drive_left_delay(600);
            run = 1;
            drive_foward_delay(100);
        }

        // if both QTI sensors detect black --> stop, turn right, and move forward

```

```
    if (left_black && right_black) {  
        drive_right_delay(900);  
        drive_foward_delay(100);  
    }  
}  
}  
}  
}  
}
```